



IS2205 - Mobile Application Design & Development

Elements of an Android Application Project

Agenda

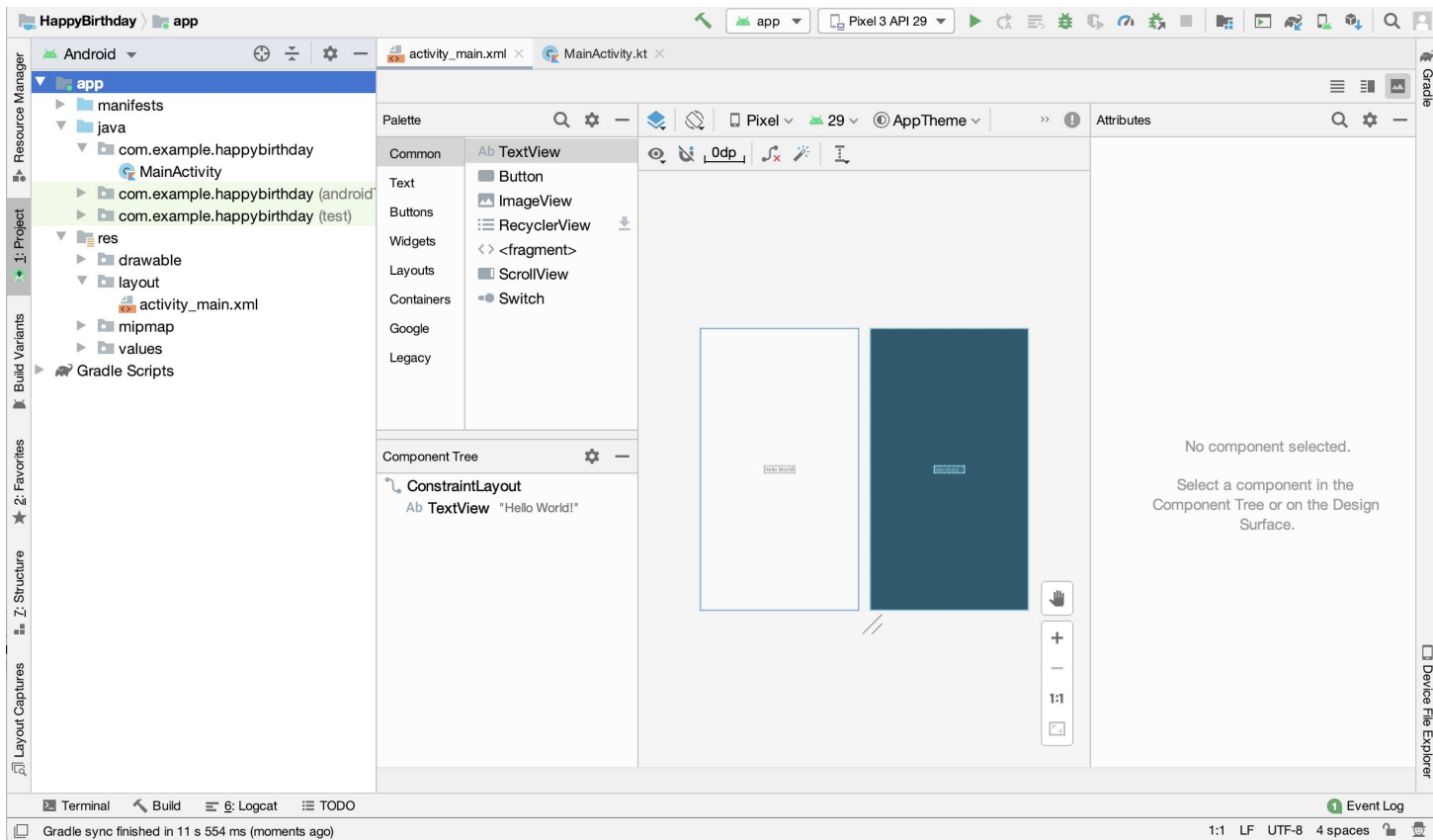
- 
- ❏ What tools you need?
 - ❏ Android app project structure
 - ❏ Layouts and resources in Android
 - ❏ Activities
 - ❏ Windows vs Activities
 - ❏ Gradle: Building an Android app

Android Studio Version

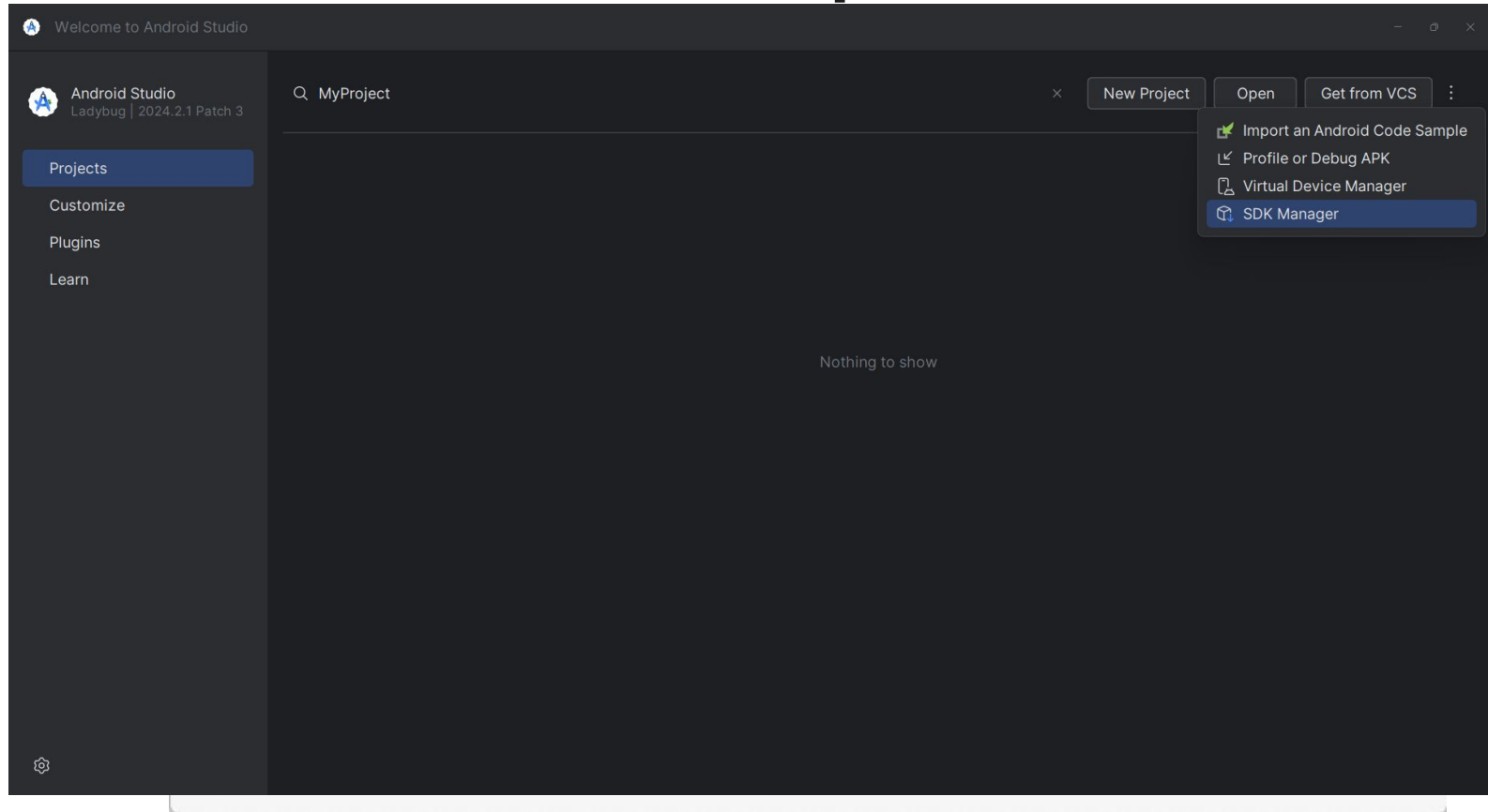


What tools you need?

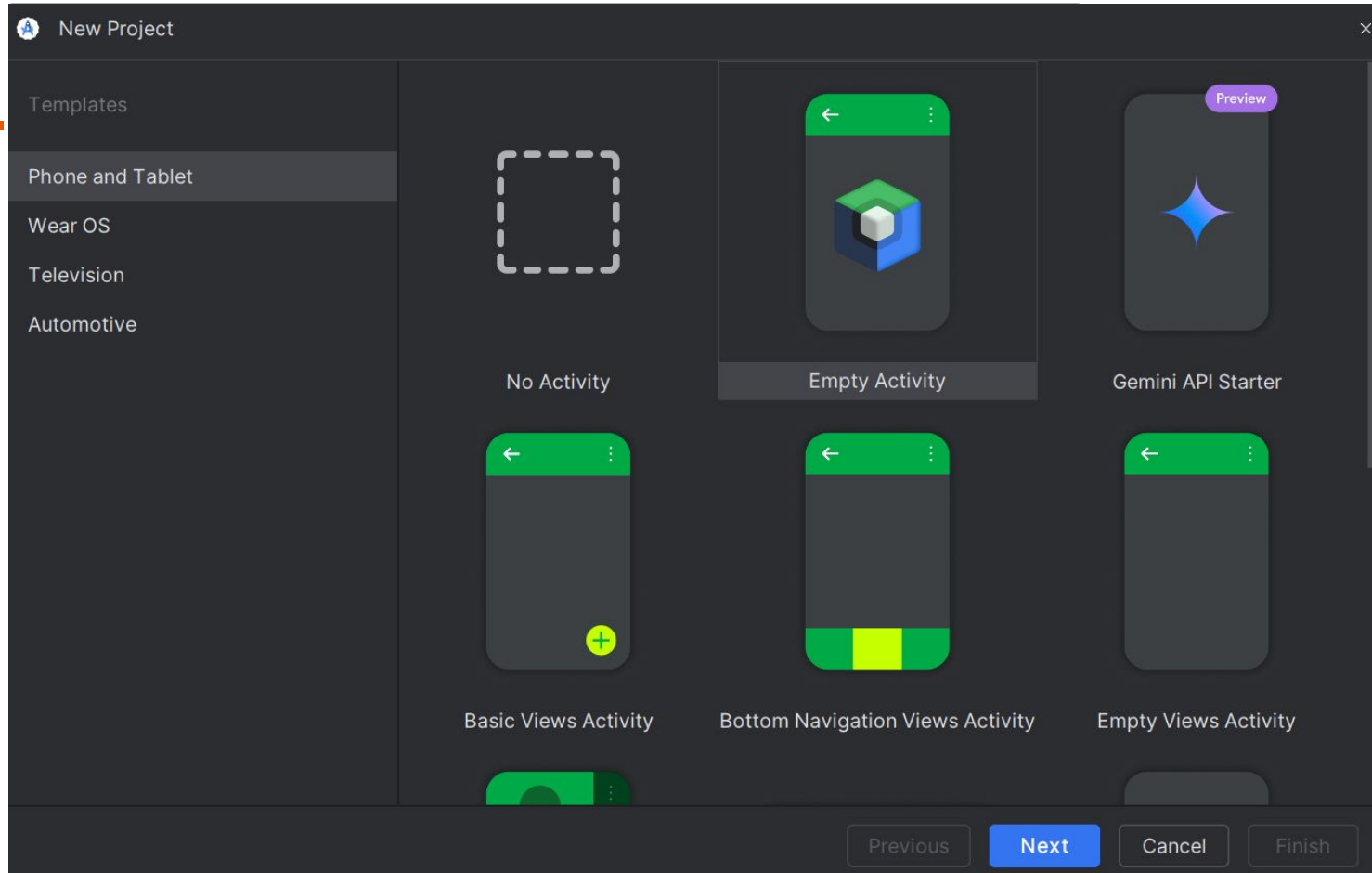
Android Studio IDE



Android Studio First Impression...



Creating a New Project



Project Preferences...

Create New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name

Package name

Save location 

Minimum SDK 

 Your app will run on approximately **97.4%** of devices.
[Help me choose](#)

Build configuration language  

Alternative names...



API 21 ("Lollipop"; Android 5.0)

API 21 ("Lollipop"; Android 5.0)

API 22 ("Lollipop"; Android 5.1)

API 23 ("Marshmallow"; Android 6.0)

API 24 ("Nougat"; Android 7.0)

API 25 ("Nougat"; Android 7.1.1)

API 27 ("Oreo"; Android 8.1)

API 28 ("Pie"; Android 9.0)

API 29 ("Q"; Android 10.0)

API 30 ("R"; Android 11.0)

API 31 ("S"; Android 12.0)

API 32 ("Sv2"; Android 12L)

API 33 ("Tiramisu"; Android 13.0)

API 34 ("UpsideDownCake"; Android 14.0)

Choose API levels for your app



- Minimum SDK: Device needs at least this API level to install
- Target SDK: API version and highest Android version tested
- Compile SDK: **Android OS library version** compiled with

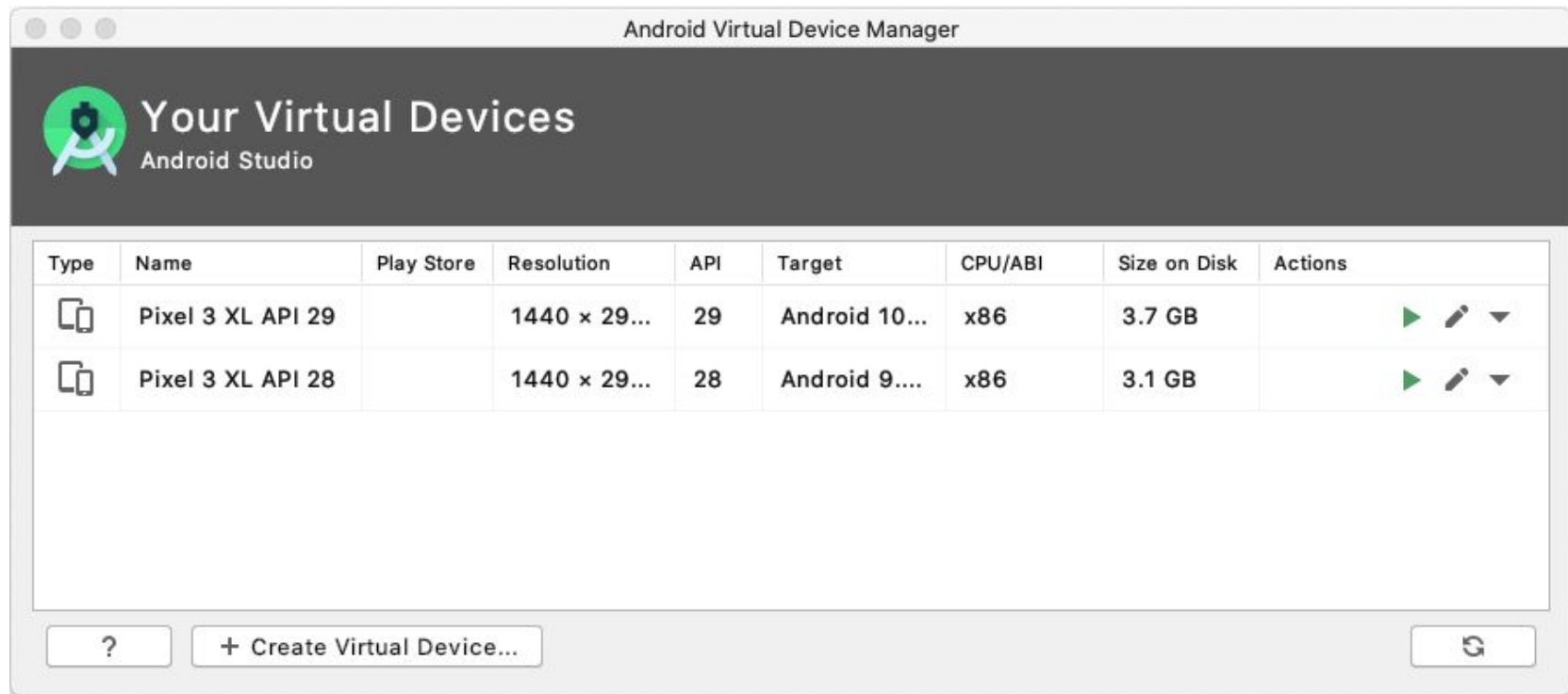
Rule: `minSdkVersion <= targetSdkVersion <= compileSdkVersion`

Running the App...



- Android device (phone, tablet)
- Emulator on your computer (Virtual Device)

Android Virtual Device (AVD) Manager



Activity



Identify 5 scenarios that Android Virtual Device will be useful for a developer than a physical phone and its data cable?

Project Content...



- ❏ Source code
- ❏ Resources (layout files, images, audio files, themes, and colors)
- ❏ Script files
- ❏ Configuration files

Project Structure in Project Directory...

Project-Directory



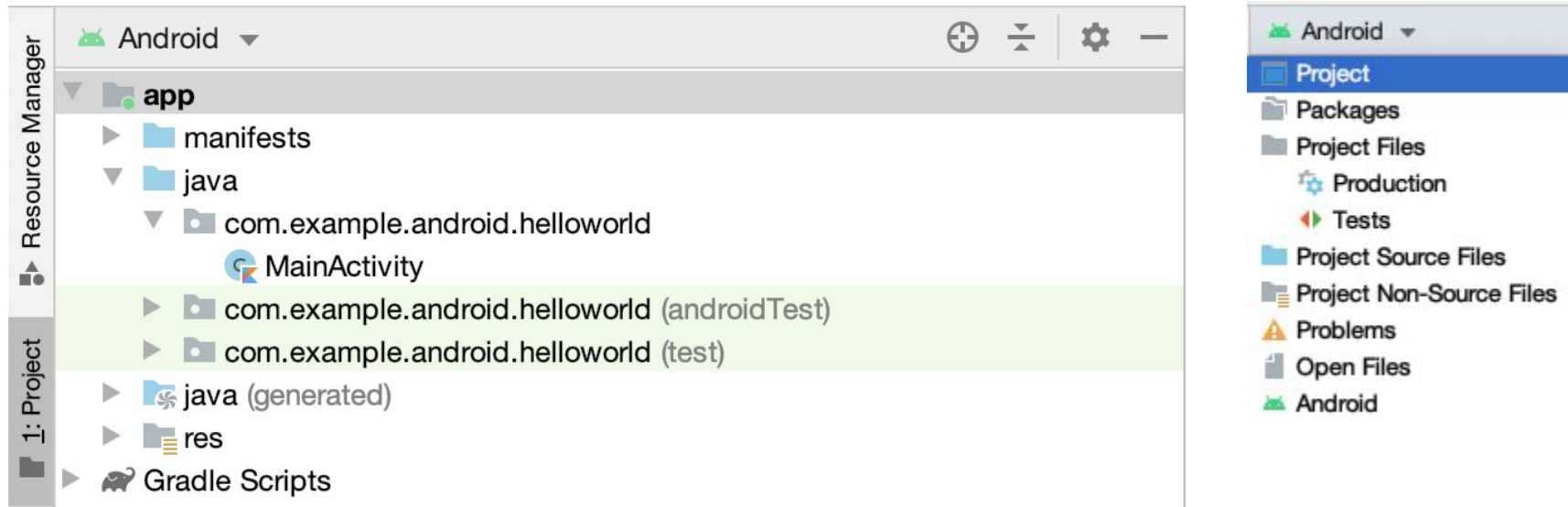
`app`: The main directory for source code, resources, and manifest file.

`build.gradle.kts(Project:modules,App)`: Kotlin build script defining dependencies, build variants etc.

`gradle.properties`: Stores Gradle configuration settings specific to this project.

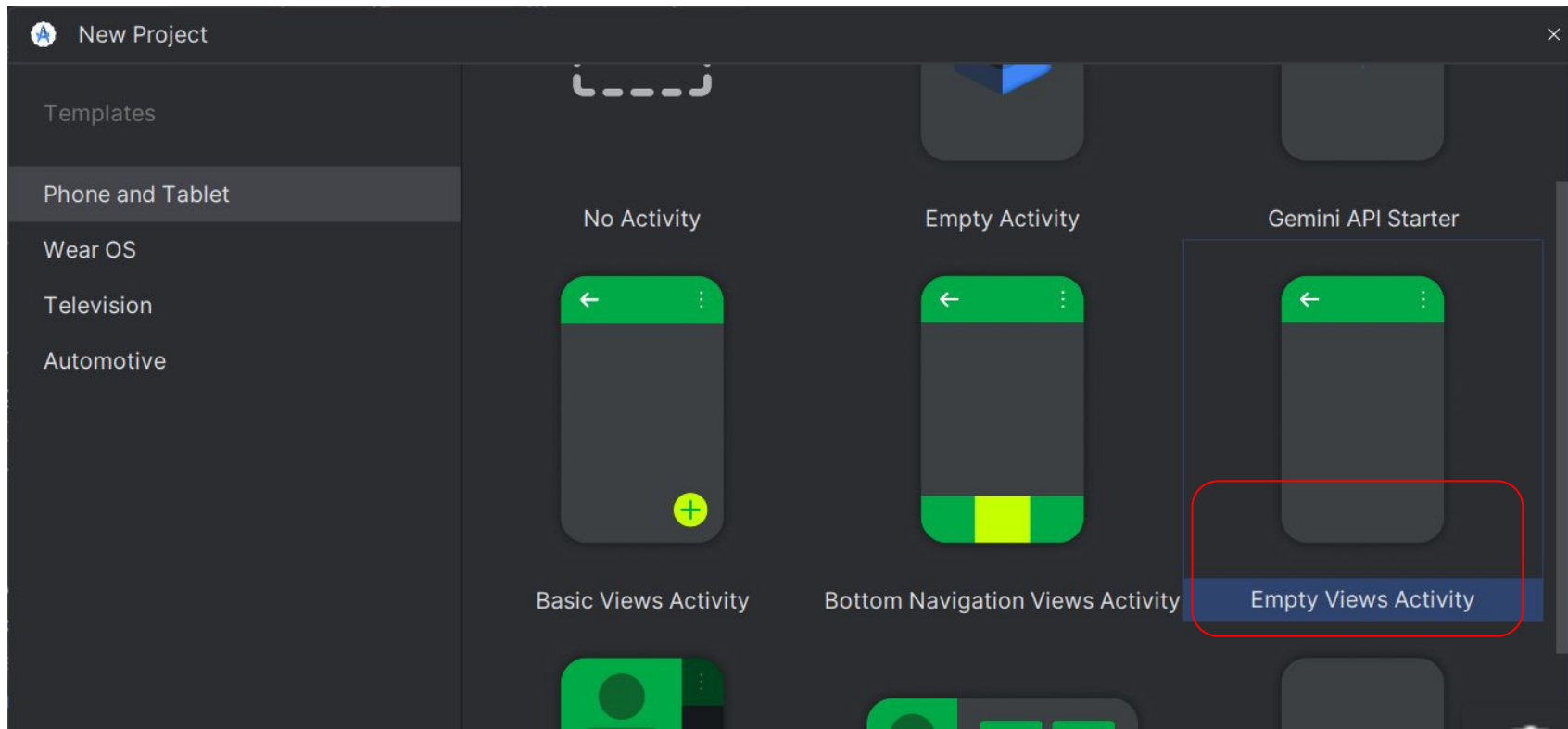
`local.properties`: Contains project-specific properties like SDK paths.

Project Files in IDE...



Views Layouts and resources in Android App Project

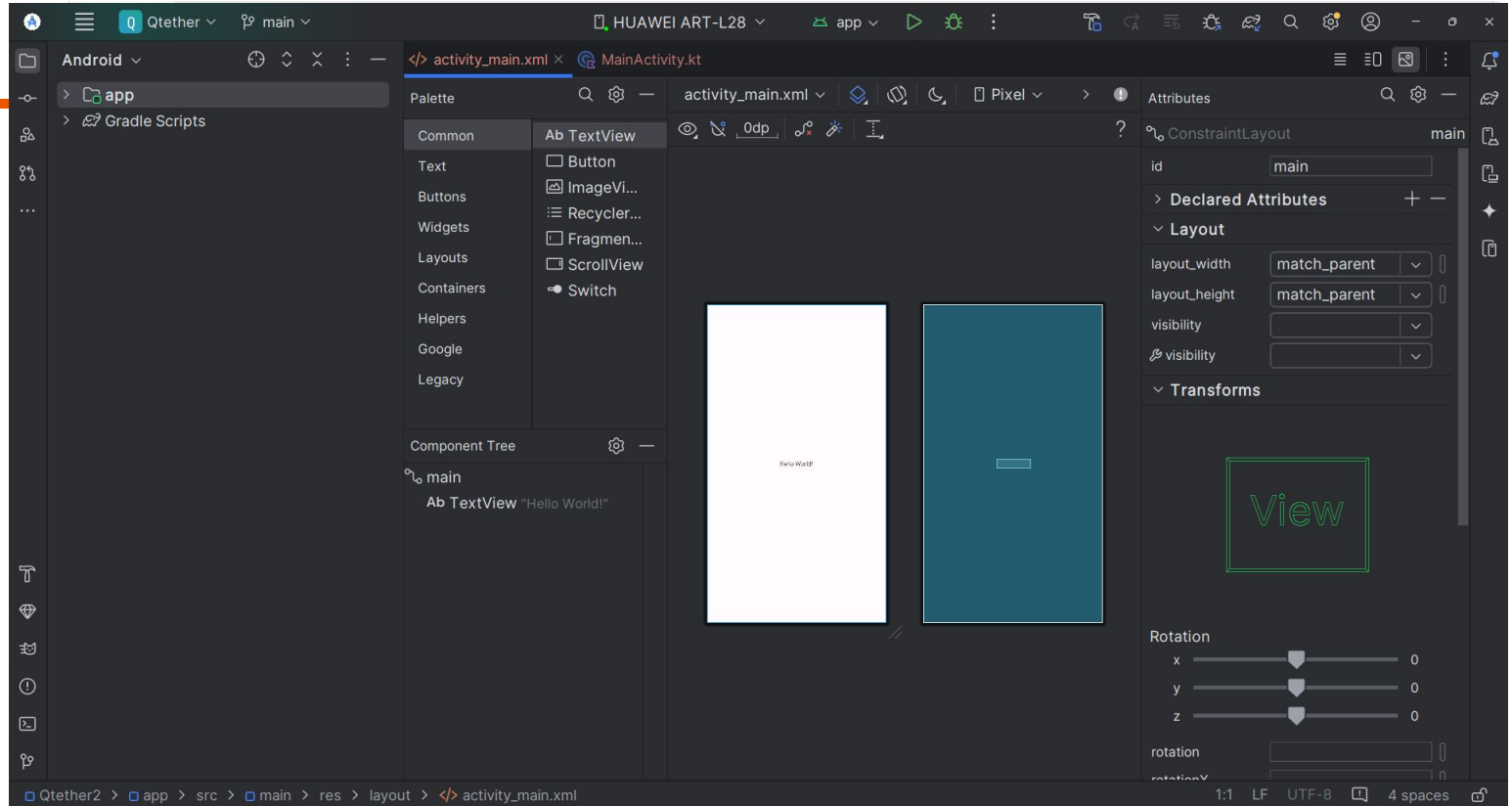
Views



Views

- ❑ **Views** are the **user interface** building blocks in Android
 - ❑ Bounded by a rectangular area on the screen
 - ❑ Responsible for drawing and event receiving
 - ❑ Examples: TextView, ImageView, Button
- ❑ Can be grouped to form more complex user interfaces

Layout Editor



XML Layouts

- ❑ You can also edit your layout as an ordinary XML(Text) file.
- ❑ Android uses XML to specify the layout of user interfaces (including View attributes)
- ❑ Each View in XML corresponds to a class in Kotlin that controls how that View functions

XML for a TextView



```
<TextView  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Hey diddle diddle!"/>
```

hey diddle diddle

Size of a View

- ❑ `wrap_content`

```
android:layout_width="wrap_content"
```

- ❑ `match_parent`

```
android:layout_width="match_parent"
```

- ❑ **Fixed value (use dp units)**

```
android:layout_width="48dp"
```

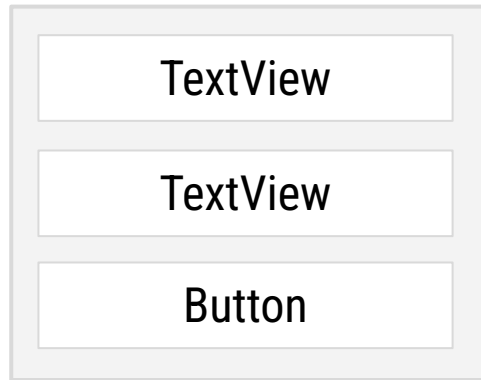
ViewGroups

A `ViewGroup` is a container that determines how views are displayed.

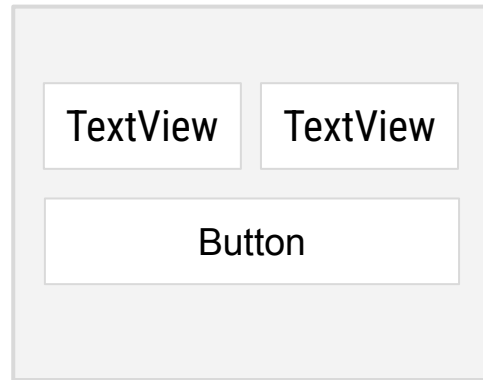
FrameLayout



LinearLayout



ConstraintLayout



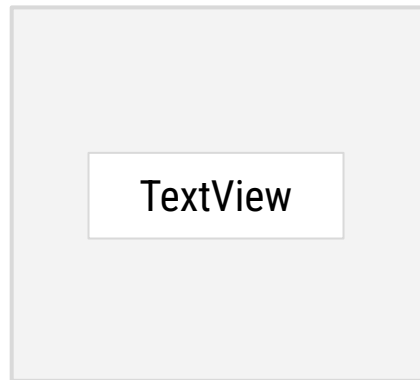
The ViewGroup is the parent and the views inside it are its children.

FrameLayout example

A `FrameLayout` normally holds a single child `View`.



```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Hello World!"/>
</FrameLayout>
```



LinearLayout example

- ❑ Aligns child views in a row or column
- ❑ Set `android:orientation` to horizontal or vertical

`<LinearLayout`

`android:layout_width="match_parent"`

`android:layout_height="match_parent"`

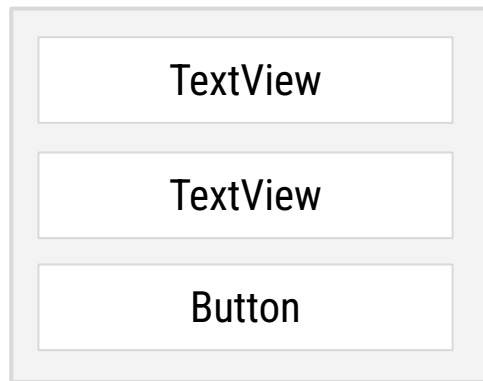
`android:orientation="vertical">`

`<TextView ... />`

`<TextView ... />`

`<Button ... />`

`</LinearLayout>`



ConstraintLayout

- ❑ Aligns child based on constraints with the layout borders or other views
- ❑ Preferred by the IDE

```
<androidx.constraintlayout.widget.ConstraintLayout ...>
```

```
    <TextView
```

```
        app:layout_constraintBottom_toBottomOf="parent"
```

```
    ... />
```

```
    <Button
```

```
        ....app:layout_constraintStart_toStartOf="parent"
```

```
        app:layout_constraintTop_toBottomOf="@+id/textView"
```

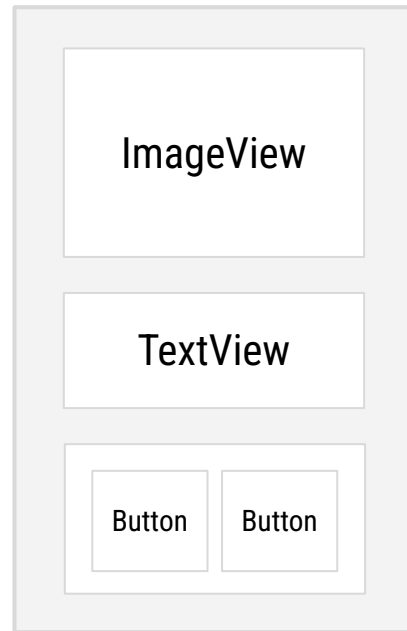
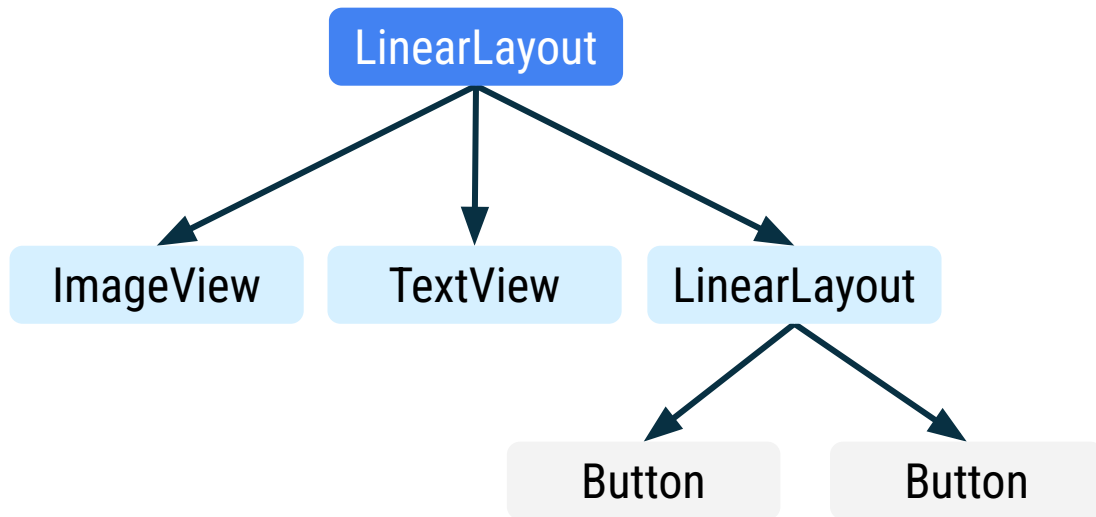
```
    />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

Reduce the
nesting

Easier to adapt
to screen sizes
and
orientations

View hierarchy



Activity 2



Using the layouting method of LinearLayout only, setup a view hierarchy to mimic the whatsapp main UI

More App Resources



Static content or additional files that your code uses

- ❏ Layout files
- ❏ Images
- ❏ Audio files
- ❏ Externalized Strings
- ❏ App icon

Common Resource Directories

Add resources to your app by including them in the appropriate resource directory under the parent `res` folder.

main

```
├── java
└── res
```

```
├── drawable - files related to drawing images and related assets
```

```
├── layout - layout XML files
```

```
└── mipmap - app icon (launcher icon) at different screen densities
```

```
└── values - collections of strings, colors, integers, and styles
```

Resource IDs

- ❑ Each resource has a resource ID to access it.
- ❑ When naming resources, the convention is to use all lowercase with underscores (for example, `activity_main.xml`).
- ❑ Android autogenerates a class file named `R.java` with references to all resources in the app.
- ❑ Individual items are referenced with:
`R.<resource_type>.<resource_name>`

Examples: `R.drawable.ic_launcher` (`res/drawable/ic_launcher.xml`)
`R.layout.activity_main` (`res/layout/activity_main.xml`)

Resource IDs for views

Individual views can also have resource IDs.

Add the `android:id` attribute to the View in XML. Use `@+id/name` syntax.

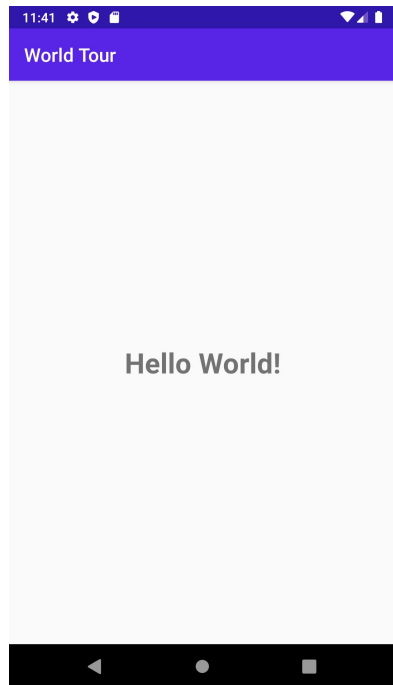
```
<TextView
    android:id="@+id/helloTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```

Within your app, you can now refer to this specific TextView using:

```
R.id.helloTextView
```

Activities

What's an Activity?



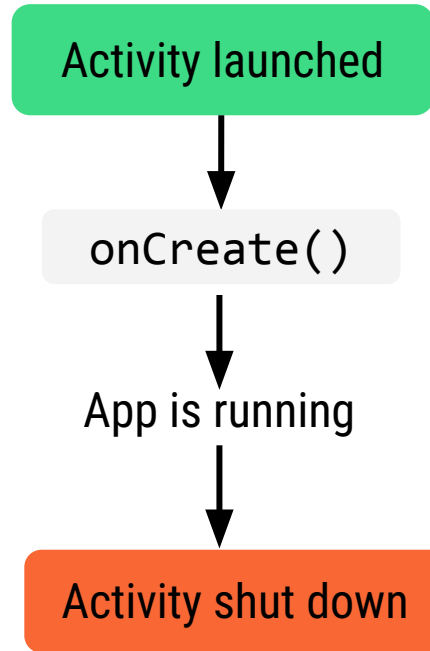
- ❏ An Activity is a means for the user to accomplish one main user need.
- ❏ An activity loosely resembles a Window in a desktop application
- ❏ An Android app is composed of one or more activities.

MainActivity.kt



```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

How an Activity runs



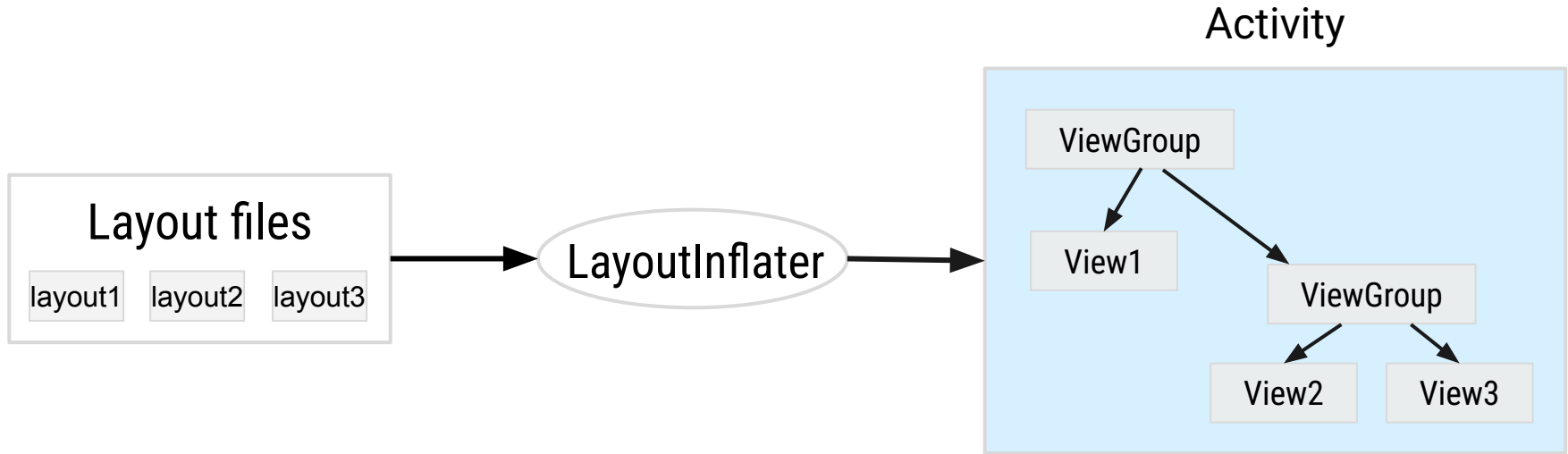
Implementing the onCreate()



Called when the system creates your Activity

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
}
```

Layout inflation



Question



What are the design differences of Android activities from Desktop windows?

Window vs Activity

	Desktop Window	Android Activity
Visual	Overlapping, resizable, and side-by-side	Typically full-screen, and stacked
Control	By the user event handlers	Constrained by the OS callbacks
Task Entry Point	A centralized application process via a main method as the entry point	Multiple activities can serve as distinct entry points for other apps.

Gradle: Building an Android app

What is Gradle?



- ❑ Build automation system
- ❑ Manages the build cycle via a series of tasks (i.e.: compiles sources, runs tests, installs app to device)
- ❑ Determines the proper order of tasks to run
- ❑ Manages dependencies between projects and third-party libraries

Gradle build file



- ❏ Declare plugins
- ❏ Define Android properties
- ❏ Handle dependencies
- ❏ Connect to repositories

Plugins



Provide libraries and infrastructure needed by the app
(Groovy)

```
apply plugin: 'com.android.application'
```

```
apply plugin: 'kotlin-android'
```

```
apply plugin: 'kotlin-android-extensions'
```

Plugins



Provide libraries and infrastructure needed by the app (Kotlin)

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
}
```

Android configuration



```
android {  
    compileSdkVersion 30  
    buildToolsVersion "30.0.2"  
    defaultConfig {  
        applicationId "com.example.sample"  
        minSdkVersion 19  
        targetSdkVersion 30  
    }  
}
```

Dependencies



```
dependencies {  
    implementation  
    "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"  
    implementation 'androidx.core:core-ktx:1.3.2'  
    implementation 'androidx.appcompat:appcompat:1.2.0'  
    implementation 'com.google.android.material:material:1.2.1'  
    ...  
}
```

Dependencies



```
dependencies {  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
    ...  
}
```


Repositories

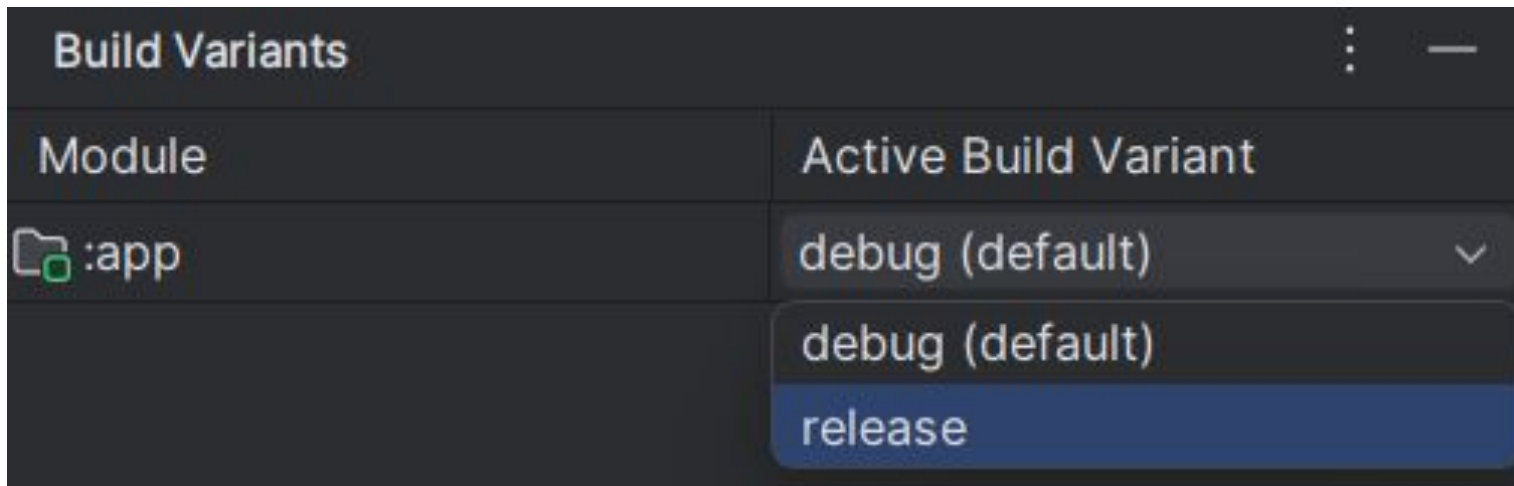


```
repositories {  
    google()  
    mavenCentral()  
}
```

Repositories



By default the IDE configures the debug variant



Common Gradle Tasks



- ❏ `gradle clean`
- ❏ `gradle build`
- ❏ `gradle tasks & tasks -all`
- ❏ `gradle installDebug`

Thank you!